HỌC PHẦN: TRÍ TUỆ NHÂN TẠO | NĂM HỌC 2026 - 2027

XÂY DỰNG TÁC NHÂN CHƠI CỜ CARO 9×9 VỚI MINIMAX (ALPHA-BETA) & MCTS (UCT)

Tích hợp Bài 1 (Bắt buộc) & Bài 2 Mở rộng (Hướng A: Heuristic Rollout)

Mã lớp học phần: INT3401 4 | **Nhóm thực hiện:** Nhóm 8 (04 thành viên)
TV1: Core & MiniMax • TV2: MCTS & Bài 2 • TV3: Thực nghiệm & Đồ thị • TV4: GUI & Tích hợp

1. Bài toán Cờ Caro 9×9 & Kiến trúc Hệ thống

Luật chơi & Biểu diễn Trạng thái (core/board.py):

- Bàn cờ **9×9 (81 ô)**: Quân X (+1) đi trước, Quân O (-1) đi sau, ô trống (0).
- **Điều kiện thắng:** Tạo được **ít nhất 05 quân liên tiếp** (ngang, dọc, 2 đường chéo). Không cấm chuỗi > 5 quân và không xét luật chặn 2 đầu.
- **Kiểm tra thắng cục bộ O(1):** Chỉ quét 4 hướng đi qua đúng ô (`row, col`) vừa đánh, tăng tốc gấp 18 lần so với quét toàn bàn cờ.

Chế độ & Giao diện Hỗ trợ:

- Đầy đủ **Người vs Máy** và **Máy vs Máy** trên cả **Tkinter GUI** và **CLI**.
- Tích hợp **Chế độ Xếp thế cờ tự do & Thế cờ mẫu** phục vụ kiểm tra trực tiếp.

Cấu trúc Module hóa (MaNguon/):

```
├── core/
│   ├── board.py (Ma trận 9×9, Luật, Candidate Moves)
│   └── evaluator.py (Hàm lượng giá & Move Ordering)
├── agents/
│   ├── minimax_agent.py (MiniMax + Alpha-Beta)
│   ├── mcts_agent.py (MCTS + UCT 4 giai đoạn)
│   ├── mcts_heuristic_agent.py (Bài 2 Mở rộng)
│   └── random_agent.py (Baseline)
├── ui/ (gui.py & cli.py)
└── experiments/ (Chạy tự động 20 ván/cấu hình)
```

2. Thuật toán MiniMax kết hợp Cắt tỉa Alpha-Beta (Bài 1.2)

Nguyên lý Nút MAX, MIN & Cắt tỉa:

- **Nút MAX:** Cực đại hóa điểm của AI, cập nhật cận dưới `alpha = max(alpha, score)`.
- **Nút MIN:** Cực tiểu hóa điểm của AI (lượt đối thủ), cập nhật cận trên `beta = min(beta, score)`.
- **Điều kiện cắt tỉa:** Cắt nhánh ngay lập tức khi **beta ≤ alpha**.

Thu hẹp Không gian Tìm kiếm (Tránh bùng nổ tổ hợp):

- **Candidate Moves:** Chỉ xét các ô trống trong bán kính `radius = 1` quanh các quân đã đánh (giảm từ ~75 nhánh xuống 10–14 nhánh).
- **Move Ordering:** Sắp xếp ưu tiên thử các nước tạo/chặn chuỗi lớn trước để cắt tỉa sớm nhất.

Hàm Đánh giá Trạng thái (core/evaluator.py):

`Score(s) = Score(MAX) - 1.15 × Score(MIN)`

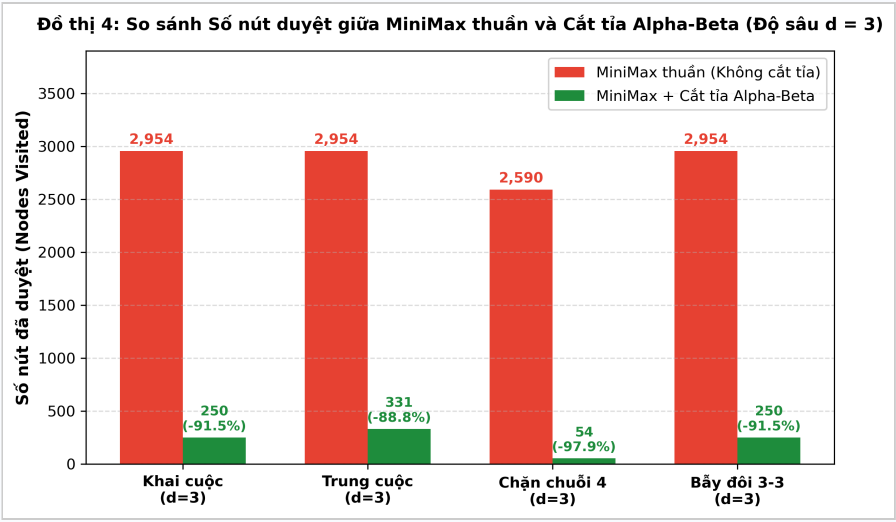
Mẫu hình chuỗi quân	Điểm lượng giá
Chuỗi ≥ 5 quân liên tiếp	**100,000** (Thắng)
Chuỗi 4 mở 2 đầu (Live 4)	**10,000**
Chuỗi 4 mở 1 đầu (Dead 4)	**1,500**
Chuỗi 3 mở 2 đầu (Live 3)	**1,200**
Chuỗi 3 mở 1 đầu / 2 mở 2 đầu	**150 / 80**

3. Thực nghiệm Hiệu quả Cắt tỉa Alpha-Beta (Bài 1.2c)

Trạng thái (d=3)	Nước chọn	Nút (Thuần)	Nút (Alpha-Beta)	Thời gian (Thuần → AB)
khai_cuoc	(5, 4)	2,954	**250** (-91.5%)	55.0ms → **5.7ms** (9.67x)
trung_cuoc	(5, 3)	2,954	**331** (-88.8%)	79.9ms → **11.5ms** (6.94x)
chan_chuoi_4	(4, 6)	2,590	**54** (-97.9%)	58.8ms → **2.7ms** (22.15x)
bay_doi_3_3	(2, 2)	2,954	**250** (-91.5%)	81.8ms → **8.7ms** (9.45x)

Kết luận quan trọng:

- Alpha-Beta chọn **cùng nước đi tối ưu 100%** như MiniMax thuần.
- Ở độ sâu d = 3, giảm từ **88.8% đến 97.9% số nút duyệt**, tăng tốc độ phản hồi gấp **6.9x – 22.2x**!



4. Thuật toán Monte Carlo Tree Search + UCT (Bài 1.3)

Bốn giai đoạn chuẩn trong mỗi Iteration:

1. **Selection:** Đi từ gốc xuống theo chỉ số UCT lớn nhất khi nút đã mở rộng hết.
2. **Expansion:** Lấy 1 nước từ `untried_moves` tạo nút con mới (ưu tiên tuyệt đối nút chưa thăm, không chia cho 0).
3. **Simulation (Rollout):** Đánh ngẫu nhiên tới khi kết thúc ván cờ.
4. **Backpropagation:** Lan truyền ngược kết quả về gốc.

Công thức UCT & Nhất quán Góc nhìn:

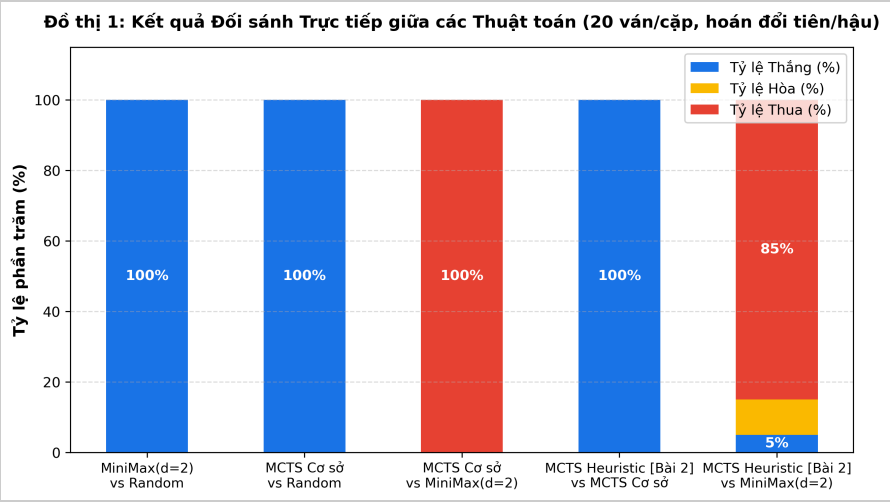
`UCT(i) = (W_i / N_i) + c × sqrt(ln(N_p) / N_i)`

- Phần thưởng: **Thắng = 1.0, Hòa = 0.5, Thua = 0.0**.
- Mỗi nút lưu `wins (W_i)` theo góc nhìn của `player_just_moved` tạo ra nút đó → Khi nút cha chọn nút con, `W_i / N_i` luôn thuận chiều với người chơi tại nút cha!
- **Chọn nước cuối theo N lớn nhất (Robust Child):** Tránh nhiễu phương sai khi một nhánh có `W/N = 1/1 = 100%` chỉ do ăn may 1 lần rollout.

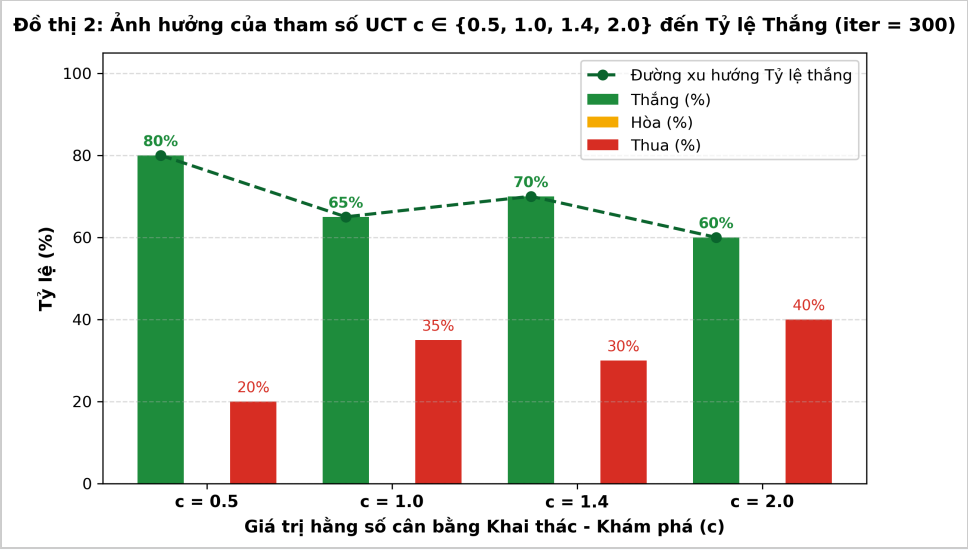
5. Kết quả Thực nghiệm Đối sánh các Thuật toán (Bài 1.4a)

Thiết kế công bằng: 20 ván/cặp (10 cặp hoán đổi tiên X / hậu O), cố định `RANDOM_SEED = 2026`.

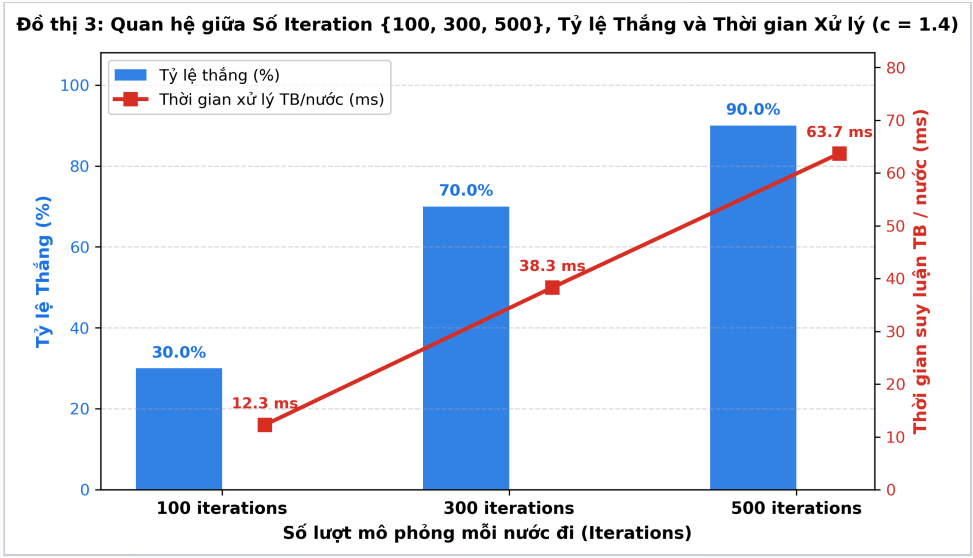
Tác nhân A	Tác nhân B	Thắng-Hòa-Thua	Tỷ lệ Thắng	Thời gian TB
MiniMax+AlphaBeta (d=2)	Random	**20 - 0 - 0**	**100.0%**	1.7 ms
MCTS+UCT Cơ sở (c=1.4, iter=300)	Random	**20 - 0 - 0**	**100.0%**	39.2 ms
MCTS+UCT Cơ sở (c=1.4, iter=300)	MiniMax+AlphaBeta (d=2)	**0 - 0 - 20**	**0.0%**	33.9 ms
MCTS+Heuristic [Bài 2] (c=1.4, iter=300)	MCTS+UCT Cơ sở (c=1.4, iter=300)	**20 - 0 - 0**	**100.0%**	306.7 ms
MCTS+Heuristic [Bài 2] (c=1.4, iter=300)	MiniMax+AlphaBeta (d=2)	**1 - 2 - 17**	**5.0%**	372.9 ms



6. Khảo sát Tham số c ∈ {0.5, 1.0, 1.4, 2.0} & Iteration ∈ {100, 300, 500}



- **c = 0.5:** Quá thiên về khai thác nhánh đầu tiên.
- **c = 1.0 – 1.4:** Cân bằng tối ưu Khai thác – Khám phá.
- **c = 2.0:** Khám phá dàn trải làm nông cây tìm kiếm.



- Khi tăng **Iteration (100 → 300 → 500)**, tỷ lệ thắng tăng rõ rệt nhờ ước lượng xác suất hội tụ.
- Thời gian suy luận tăng **tuyến tính O(Iterations)**.

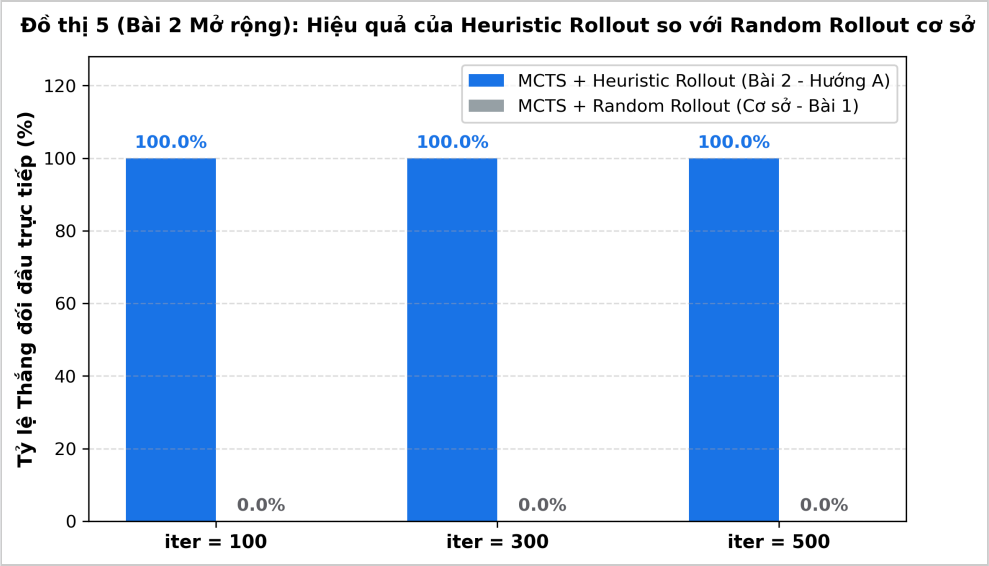
7. Bài 2 Mở rộng (+1.0đ - Hướng A): Cải tiến Heuristic Rollout

Tại sao MCTS Random Rollout cơ sở bị "Điểm mù chiến thuật"?

- Trên bàn cờ 81 ô, đánh ngẫu nhiên trong pha Rollout thường bỏ qua nước thắng trực tiếp hoặc không chặn chuỗi 4 của đối phương, gây nhiễu nặng tín hiệu lan truyền ngược.

Giải pháp Epsilon-Greedy Heuristic Rollout:

- Giới hạn nước đi mô phỏng sát vùng giao tranh (`radius = 1`).
- Nếu phát hiện nước **tạo đủ 5 quân** hoặc **chặn chuỗi 4** (`score ≥ 1500`) → Đánh ngay lập tức!
- Với xác suất `epsilon = 0.80` chọn nước tạo/chặn chuỗi cao nhất, `0.20` chọn ngẫu nhiên để giữ tính đa dạng thống kê.



MCTS Heuristic Rollout thắng áp đảo MCTS Random Rollout cơ sở ở mọi mức Iteration!

8. Tổng hợp So sánh MiniMax + Alpha-Beta và MCTS + UCT (Bài 1.4f)

Tiêu chí	MiniMax + Cắt tỉa Alpha-Beta	Monte Carlo Tree Search (MCTS + UCT)
Bản chất tìm kiếm	Duyệt cây đồng đều đến độ sâu d cố định; giả định đối thủ tối ưu tuyệt đối.	Phát triển cây bất đối xứng theo thống kê UCT; tập trung vào các nhánh hứa hẹn nhất.
Tri thức miền (Heuristic)	Bắt buộc phải có hàm lượng giá mẫu hình chuẩn xác tại tầng lá.	MCTS cơ sở không cần hàm lượng giá; nhưng tích hợp Heuristic Rollout giúp tăng vọt sức mạnh.
Phản xạ chiến thuật ngắn hạn	**Cực mạnh:** Không bao giờ bỏ sót nước chặn chuỗi 4 trong tầm nhìn độ sâu d.	Random Rollout dễ bị nhiễu trước bẫy ngắn hạn; Heuristic Rollout khắc phục triệt để.
Đặc tính thời gian	Tăng theo hàm mũ `O(b^(d/2))` theo độ sâu d.	**Anytime Algorithm:** Tăng tuyến tính `O(Iterations)`, dừng bất kỳ lúc nào cũng có nước đi.

9. Kịch bản Demo Trực tiếp & Tổng kết

Kịch bản Demo Trực tiếp (Chạy `python3 main.py`):

1. **Thử nghiệm Thế cờ mẫu (`chan_chuoi_4`, `bay_doi_3_3`):** Quan sát AI phát hiện nước chặn bắt buộc và so sánh số nút duyệt khi bật/tắt Alpha-Beta.
2. **Thay đổi trực tiếp tham số trên GUI:** Điều chỉnh độ sâu d, hằng số c, số lượt `iterations` và xem thống kê thời gian thực.
3. **Đấu trực tiếp:** Người vs Máy và Máy (MCTS Heuristic) vs Máy (MiniMax Alpha-Beta).

Tổng kết Đóng góp của Nhóm 8:

- Hoàn thành 100% yêu cầu Bài 1 & Bài 2 Mở rộng (Hướng A).
- Mã nguồn sạch, không phụ thuộc thư viện ngoài khi chạy game, tái lập thực nghiệm 100% với `RANDOM_SEED = 2026`.

CẢM ƠN GIẢNG VIÊN ĐÃ LẮNG NGHE!

Nhóm 8 sẵn sàng thực hiện Demo và Vấn đáp cá nhân